Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»
КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ

НА ТЕМУ:

«Система управления реестром docker-образов на основе системы Harbor»

Студент	**ИУ7-53Б** (Группа)	(Подпись, дата)	**А. К. Пресняков** (И. О. Фамилия)
Руководитель НИР		(Подпись, дата)	**А. П. Ковтушенко** (И. О. Фамилия)
Консультант		(Подпись, дата)	**Ю. В. Строганов** (И. О. Фамилия)
Нормоконтролёр		(Подпись, дата)	(И. О. Фамилия)

2025 г.

РЕФЕРАТ

Научно-исследовательская работа 28 с., 5 рис., 3 табл., 45 ист., 2 прил. Объект исследования – системы управления реестром docker-образов.

Цель работы – проведение обзора и сравнительного анализа современных систем управления реестрами docker-образов с детальным рассмотрением платформы Harbor как эталонного решения.

СОДЕРЖАНИЕ

РЕФЕРАТ **3**

ВВЕДЕНИЕ **5**

1 Анализ предметной области **6**

1.1 Современное состояние контейнеризации и роль реестров образов 6

1.2 Понятия Docker и технологии контейнеризации 7

1.2.1 Основы контейнеризации и Docker 7

1.2.2 Реестры контейнерных образов и их компоненты 8

1.2.3 Безопасность и управление в реестрах 9

1.2.4 Стандарты и спецификации 9

1.3 Классификация задач управления реестром контейнерных образов 10

1.3.1 Задачи безопасности и соответствия требованиям 10

1.3.2 Задачи управления жизненным циклом артефактов 10

1.3.3 Задачи обеспечения доступности и производительности 11

1.3.4 Задачи интеграции и операционного управления 11

1.3.5 Система Harbor как эталонное решение 12

1.4 Практическое развёртывание системы Harbor 14

2 Существующие решения **16**

2.1 Критерии сравнения систем управления реестром 16

2.2 Результаты сравнения 17

2.3 Анализ полученных результатов 17

2.4 Интеграция Harbor в учебный процесс кафедры 18

2.4.1 Сценарий использования для лабораторных работ 19

2.4.2 Пример реализации CI/CD пайплайна 20

ЗАКЛЮЧЕНИЕ **22**

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ **23**

ПРИЛОЖЕНИЕ А **26**

ПРИЛОЖЕНИЕ Б **27**

ВВЕДЕНИЕ

Управление реестрами docker-образов – одна из основных технологий в области контейнеризации и DevOps, обеспечивающая надежное хранение, распространение и контроль версий контейнерных артефактов. В условиях повсеместного перехода на микросервисную архитектуру и облачные технологии, вопросы эффективного управления образами становятся критически важными для обеспечения непрерывной интеграции и доставки программного обеспечения.

Актуальность использования систем управления реестрами обусловена необходимостью создания безопасных и отказоустойчивых конвейеров поставки ПО, а также предотвращения уязвимостей в цепочке поставок программного обеспечения.

Одним из наиболее перспективных решений для управления реестрами является система Harbor. Harbor представляет собой открытый реестр контейнерных образов корпоративного уровня, который обеспечивает комплексную безопасность, включая сканирование уязвимостей, цифровую подпись образов и управление доступом на основе ролей. В отличие от базовых реестров, Harbor предоставляет расширенные функции репликации, управления политиками хранения и поддержки различных типов артефактов, что делает его оптимальным решением для крупных организаций [16].

Цель работы – провести сравнительный анализ современных систем управления реестрами docker-образов на основе платформы Harbor.

Для выполнения поставленной цели необходимо выполнить следующие задачи:

1) изучить основные понятия и задачи управления реестрами контейнерных образов;
2) проанализировать существующие программные решения для управления реестрами;
3) выделить ключевые критерии для сравнительной оценки систем управления реестрами;
4) провести сравнительный анализ решений на основе сформулированных критериев;
5) определить преимущества и ограничения системы Harbor в сравнении с аналогами;
6) дать оценку поддержки критериев среди сущетсвующих систем.

1 Анализ предметной области

1.1 Современное состояние контейнеризации и роль реестров образов

Контейнеризация стала фундаментальной технологией современной IT-индустрии, кардинально изменившей подходы к разработке, поставке и эксплуатации программного обеспечения. Согласно исследованиям, более 70% организаций используют контейнеры в среде разработки крупных сервисов, причем рост адаптации составляет порядка 20% в год [14]. Стремительный рост обусловлен преимуществами, которые предоставляют контейнерные технологии: изоляция приложений, переносимость, эффективное использование ресурсов и ускорение процессов развертывания.

На рисунке 1.1 представлена диаграмма использования контейнерых приложений крупными компаниями в 2024 году.

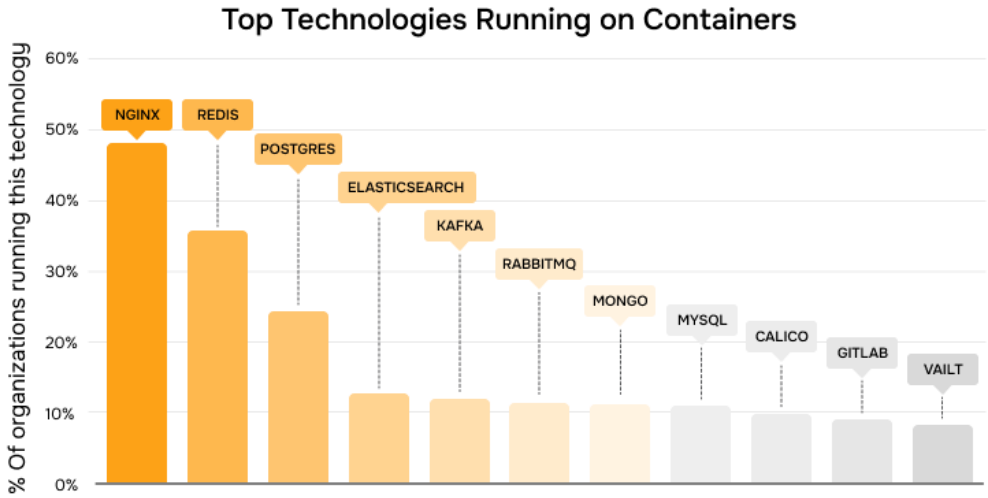


Рисунок 1.1 — Диаграмма использования контейнерых приложений крупными компаниями в 2024 году

Технология Docker стала стандартом в области контейнеризации [10] – она предоставила разработчикам простой и унифицированный интерфейс упаковки приложений со всеми зависимостями, что решило проблему совместимости на всех этапах жизненного цикла ПО.

В микросервисной архитектуре, которая доминирует в современных облачных приложениях, количество контейнерных образов может достигать тысяч и десятков тысяч. Каждый сервис, каждая его версия требуют отдельного образа. Управление этим множеством артефактов становится критически важной задачей, сравнимой по сложности с управлением исходным кодом [12].

Роль реестров контейнерных образов в этой экосистеме невозможно переоценить. Реестры выполняют функции, аналогичные репозиториям исходного кода, но для бинарных артефактов. Они обеспечивают:

1) централизованное хранение – единый источник данных для всех контейнерных образов организации;

2) версионирование и контроль изменений – возможность отслеживать историю изменений образов;

3) распределение артефактов – эффективную доставку образов до сред выполнения;

4) контроль доступа – разграничение прав на чтение и запись различных категорий пользователей.

Особую значимость реестры образов приобретают в контексте DevOps практик и процессов непрерывной интеграции и доставки (CI/CD). Современные CI/CD пайплайны генерируют десятки и сотни новых версий образов ежедневно, и без надежной системы управления этими артефактами эффективная работа становится невозможной [11].

Проблема безопасности цепочки поставок ПО также выдвигает дополнительные требования к системам управления реестрами. После ряда громких инцидентов, связанных с компрометацией контейнерных образов, организации осознали необходимость внедрения строгого контроля целостности, сканирования уязвимостей и цифровой подписи образов [15].

Рынок систем управления реестрами демонстрирует активный рост. Согласно анализу, ежегодный прирост рынка составляет около 25%, причем наибольшую динамику показывают решения с открытым исходным кодом, которые сочетают в себе корпоративный функционал и гибкость развертывания [13].

Таким образом, системы управления реестрами контейнерных образов превратились из простых хранилищ в сложные платформы, обеспечивающие безопасность, наблюдаемость и управляемость всего жизненного цикла контейнерных артефактов. Их роль в современных ИТ-ландшафтах продолжает усиливаться по мере роста масштабов использования контейнерных технологий.

1.2 Понятия Docker и технологии контейнеризации

1.2.1 Основы контейнеризации и Docker

Контейнеризация представляет собой технологию изоляции приложений на уровне операционной системы, позволяющую запускать процессы в изолированных пользовательских пространствах (контейнерах), разделяющих ядро хостовой системы. В основе современных технологий контейнеризации лежит концепция docker-образа - неизменяемого шаблона, состоящего из слоев, доступных только для чтения, каждый из которых описывает определенное изменение файловой системы [10]. Эти слои обеспечивают эффективное хранение и распространение приложений, поскольку общие слои могут использоваться несколькими образами. Для внесения изменений создаются собственные надстройки, которые записываются в самый верхний слой, который доступен для записи:

Docker-контейнер представляет собой запущенный экземпляр образа, изолированный от

других контейнеров и хост-системы [2]. Изоляция достигается за счет использования возможностей ядра Linux, таких как namespaces и cgroups. Важным свойством контейнеров является их переносимость: образ, собранный на одной системе, будет идентично работать на любой другой системе с поддержкой Docker.

На рисунке 1.2 представлена структура Docker-образа.

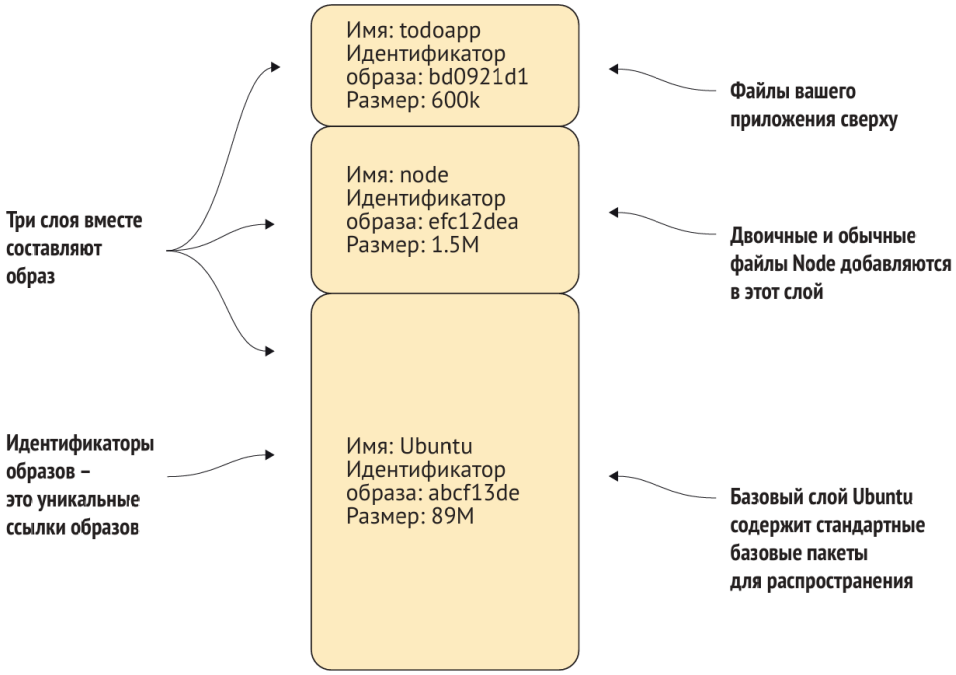


Рисунок 1.2 — Структура Docker-образа: базовый образ, слои приложений и контейнерный слой

1.2.2 Реестры контейнерных образов и их компоненты

Системы управления реестрами образуют центральный элемент инфраструктуры контейнеризации. Реестр Docker-образов функционирует как специализированное хранилище, предоставляющее REST API для операций загрузки и скачивания образов [2]. Внутри реестра образы организованы в репозитории - логические группы, обычно соответствующие отдельным приложениям или сервисам.

Каждый репозиторий содержит одну или несколько версий образа, идентифицируемых с помощью тегов. Тегирование позволяет управлять версиями артефактов и обеспечивает гибкость в развертывании приложений [16]. Помимо Docker-образов, современные реестры поддерживают хранение различных типов артефактов. К таким артефактам относятся, например, Helm-чарты – пакеты, описывающие набор ресурсов Kubernetes для развёртывания приложений, а также любые OCI-совместимые (Open Container Initiative) объекты, соответствующие открытому стандарту формата контейнеров и артефактов. Эта возможность превращает реестр из узкоспециализированного хранилища в универсальную платформу для управления бинарными артефактами [9].

1.2.3 Безопасность и управление в реестрах

В контексте корпоративного использования реестры образов должны обеспечивать комплексные механизмы безопасности. Управление доступом на основе ролей (RBAC) позволяет гибко настраивать политики доступа для различных категорий пользователей. Интеграция с корпоративными системами аутентификации, такими как LDAP (Lightweight Directory Access Protocol) — протокол для доступа к службам каталогов, и OIDC (OpenID Connect) — современный протокол аутентификации поверх OAuth 2.0, обеспечивает единое (централизованное) управление учетными записями и соответствие корпоративным стандартам безопасности.

Сканирование уязвимостей стало неотъемлемой функцией современных реестров. Этот процесс предполагает автоматический анализ компонентов образа на наличие известных уязвимостей и предоставление отчетов о безопасности [15]. Для обеспечения целостности и доверия к артефактам применяется цифровая подпись образов, гарантирующая их подлинность и неизменность.

Репликация между экземплярами реестра обеспечивает отказоустойчивость и географическое распределение, что особенно важно для организаций с распределенной инфраструктурой [7]. Механизмы репликации могут быть синхронными или асинхронными, в зависимости от требований к согласованности данных и задержкам.

1.2.4 Стандарты и спецификации

Развитие экосистемы контейнеризации привело к появлению стандартов, обеспечивающих совместимость между различными инструментами. Спецификация OCI (Open Container Initiative) определяет стандартные форматы контейнерных образов и сред выполнения [3]. Этот стандарт обеспечивает переносимость образов, созданных с помощью Docker, в другие OCI-совместимые системы контейнеризации (Podman, containerd), способствуя интероперабельности инструментов экосистемы.

API Docker Registry стал стандартом для базового взаимодействия с реестрами образов. Однако современные корпоративные решения, такие как Harbor, расширяют его возможности, добавляя функции, критически важные для организаций: управление доступом на основе ролей (RBAC), сканирование образов на уязвимости, георепликацию, управление политиками хранения и поддержку разнородных артефактов (Helm-чарты, OCI) [16]. Поддержка как базового стандарта, так и его расширений обеспечивает интероперабельность между различными инструментами экосистемы контейнеризации, такими как оркестраторы (Kubernetes), системы CI/CD (Jenkins, GitLab CI) и сканеры безопасности (Trivy, Clair).

1.3 Классификация задач управления реестром контейнерных образов

1.3.1 Задачи безопасности и соответствия требованиям

Безопасность является критически важным аспектом управления реестрами образов, особенно в свете участившихся инцидентов, связанных с цепочкой поставок ПО. Реальные атаки демонстрируют различные векторы угроз: внедрение вредоносного кода в базовые образы, компрометация учетных записей с правами на запись в реестр и использование уязвимостей в самих системах управления реестрами [15].

Аутентификация и авторизация системы должны предоставлять гибкие механизмы управления доступом, включая интеграцию с корпоративными системами каталогов (LDAP, Active Directory) и поддержку стандартов OIDC, OAuth 2.0. Управление доступом на основе ролей (RBAC) позволяет точно настраивать права для различных категорий пользователей, что является одним из самых строгих критериев безопасности управления [16].

На рисунке 1.3 представлена схема стандртов ролевого подхода к управлению доступом.

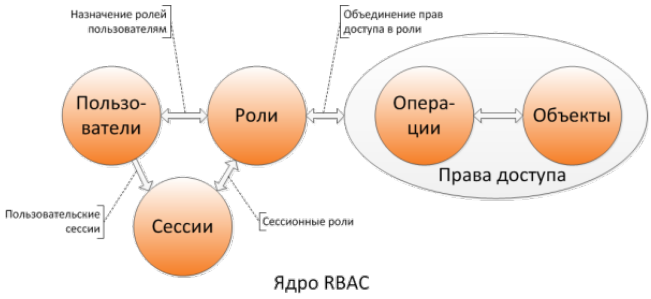


Рисунок 1.3 — Стандарты ролевого подхода к управлению доступом

Кроме этого, необходимо введение автоматического обнаружения известных уязвимостей в контейнерных образах стало обязательным требованием для корпоративных реестров [8]. Современные системы интегрируют различные сканеры и предоставляют детальные отчеты о безопасности.

Цифровая подпись и проверка целостности необходимы для обеспечения доверия к артефактам. Для этого используются специализированные механизмы, как например Docker Content Trust – встроенная в Docker Engine система, использующая набор ключей для подписи и проверки образов [2]. Эта система применяют криптографические методы для создания цифровых подписей, что гарантирует, что образы не были изменены после их публикации и получены из доверенного источника.

1.3.2 Задачи управления жизненным циклом артефактов

Эффективное управление жизненным циклом контейнерных образов необходимо для поддержания порядка в реестре и оптимизации использования ресурсов. Системы должны

предоставлять удобные механизмы для управления версиями образов через теги. Это включает поддержку семантического версионирования (Semantic Versioning, SemVer) — схемы, где номер версии структурированно указывает на характер изменений (мажорные, минорные, патчи), что помогает автоматизировать процессы обновления. Также необходима гибкая работа с тегами (метками), позволяющая, помимо версионных, использовать плавающие теги (latest, stable) и теги для различных сред (prod, staging) [2].

Автоматическое удаление устаревших образов на основе настраиваемых правил помогает экономить дисковое пространство, что особенно актуально при больших объемах хранимых данных [16]. Установка лимитов на использование дискового пространства для проектов или пользователей предотвращает неконтролируемый рост объема данных и обеспечивает равномерное распределение ресурсов [7]. Помимо Docker-образов, современные реестры должны поддерживать Helm-чарты, OPA-политики и другие OCI-артефакты, что делает их универсальными платформами для управления бинарными артефактами в облачной экосистеме [9].

1.3.3 Задачи обеспечения доступности и производительности

Для обеспечения бесперебойной работы систем, использующих реестр образов, необходимо решать комплекс задач, связанных с доступностью и производительностью.

Синхронизация данных между несколькими сущностями реестра обеспечивает отказоустойчивость и снижает задержки для распределенных команд, при этом репликация может быть синхронной или асинхронной в зависимости от требований к согласованности [7]. Кэширование часто запрашиваемых образов ускоряет их получение и снижает нагрузку на внешние реестры, а проксирование запросов к публичным реестрам позволяет контролировать внешний трафик и обеспечивать дополнительный уровень безопасности [16].

Возможность горизонтального масштабирования системы под растущую нагрузку является обязательным требованием для крупных организаций, что позволяет адаптироваться к увеличению числа пользователей и объема хранимых данных [12]. Регулярное создание бэкапов метаданных и образов обеспечивает возможность быстрого восстановления после сбоев, минимизируя простои и потери данных в критических ситуациях [15].

1.3.4 Задачи интеграции и операционного управления

Интеграция с существующей инфраструктурой и удобство операционного управления существенно влияют на успешность внедрения системы управления реестром. Поддержка webhook-ов и API для автоматизации процессов сборки и развертывания позволяет интегрировать реестр в процессы непрерывной интеграции и доставки, что ускоряет разработку и повышает ее надежность [10].

Сбор метрик использования, логов операций и ведение аудиторского журнала обеспечивают возможность анализа работы системы и расследования инцидентов безопасности. Удобный веб-интерфейс для просмотра образов, управления правами доступа и анализа отчетов безопас-

ности значительно упрощает ежедневную работу администраторов и разработчиков, снижая порог вхождения и уменьшая вероятность ошибок. Программные интерфейсы и инструменты командной строки предоставляют возможности для автоматизации задач управления и интеграции со скриптами, что позволяет создавать сложные сценарии работы с реестром и включать его в более крупные автоматизированные процессы.

1.3.5 Система Harbor как эталонное решение

Система Harbor представляет собой открытый реестр контейнерных образов корпоративного уровня, разрабатываемый под эгидой Cloud Native Computing Foundation (CNCF). Изначально созданный компанией VMware, Harbor быстро стал одним из наиболее популярных решений для управления артефактами в облачной экосистеме благодаря своей комплексной функциональности и активному сообществу contributors [16].

Архитектура Harbor построена вокруг модульной структуры, включающей основные компоненты: основные службы, отвечающие за базовые операции с образами, службы реестра, реализующие Docker Registry API, предоставляющий веб-интерфейс, дополнительные службы безопасности и тегирования для цифровой подписи образов [16]. Остальные службы предназначены для удобной интеграции с другими сервиса контейнеризации, согласно стандарту OCI. Такая архитектура обеспечивает гибкость развертывания и возможность масштабирования отдельных компонентов в зависимости от нагрузки.

На рисунке 1.4 представлена архитектура системы управления реестром docker-образов Harbor.

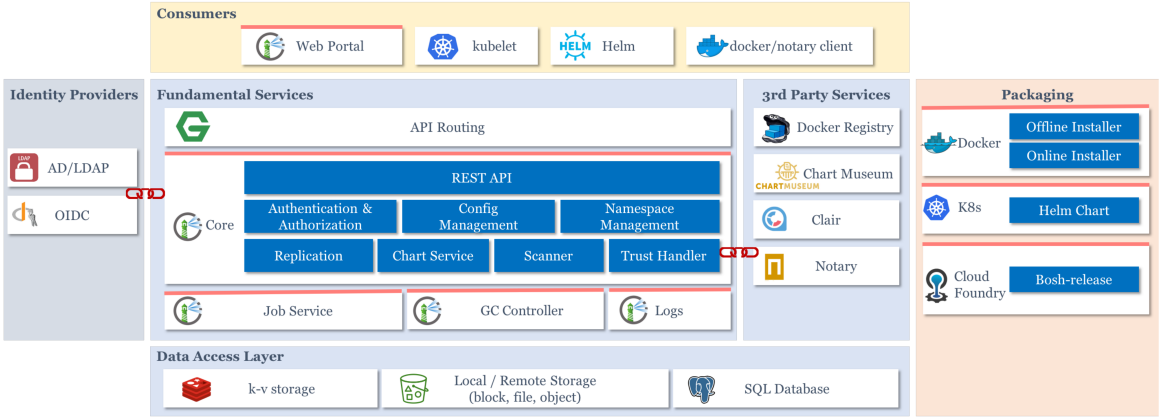


Рисунок 1.4 — Архитектура системы управления реестром docker-образов Harbor

Ключевой особенностью Harbor является встроенная поддержка механизмов безопасности, которые обычно требуют отдельной настройки в других решениях. Система включает интегрированное сканирование уязвимостей с использованием Clair, что позволяет автоматически анализировать образы на наличие известных уязвимостей сразу после их загрузки в реестр. Механизм цифровой подписи гарантирует целостность и аутентичность артефактов, предотвращая использование модифицированных образов в среде разработки [16].

Управление доступом в Harbor реализовано через ролевую модель (RBAC) с возможностью интеграции с внешними системами аутентификации, включая LDAP, Active Directory и OIDC-провайдеры. Это позволяет крупным организациям использовать существующую инфраструктуру управления идентификацией и соблюдать корпоративные политики безопасности [7]. Система поддерживает мультизадачность через проекты, где каждый проект может иметь собственные настройки доступа, политики хранения и квоты.

Функциональность репликации в Harbor обеспечивает синхронизацию образов между различными состояниями реестра, что критически важно для организаций с распределенной инфраструктурой. Поддержка как push, так и pull репликации позволяет гибко настраивать стратегии распределения артефактов между дата-центрами и облачными провайдерами [4]. В сочетании с возможностями кэширования и проксирования внешних реестров это делает Harbor эффективным решением для построения распределенной системы управления артефактами.

Harbor также выделяется поддержкой различных типов артефактов помимо Docker-образов. Система может хранить Helm-чарты, OPA-политики и другие OCI-совместимые объекты, что превращает ее в универсальную платформу для управления всеми типами бинарных артефактов в cloud-native среде [9]. Интеграция с популярными CI/CD системами через webhook-и и REST API обеспечивает seamless встраивание в процессы непрерывной интеграции и доставки.

Благодаря перечисленным характеристикам, Harbor является идеальным эталоном для сравнительного анализа систем управления реестрами. Его функциональность охватывает все ключевые аспекты, выделенные в предыдущих разделах: безопасность, управление жизненным циклом, обеспечение доступности и интеграционные возможности.

1.4 Практическое развёртывание системы Harbor

Для проверки функциональных возможностей, рассмотренных в предыдущих разделах, и для использования в качестве корпоративного реестра требуется корректное развёртывание системы Harbor. В данном подразделе представлен базовый порядок установки Harbor на операционную систему Ubuntu Server 22.04 LTS, который может быть адаптирован для других дистрибутивов Linux. Развёртывание описано в тестовом режиме; для производственной среды требуется дополнительная настройка, включая использование SSL-сертификатов, внешней базы данных и объектного хранилища [16].

1) аппаратные требования: минимум 4 ГБ ОЗУ, 2 ядра CPU, 40 ГБ дискового пространства для работы системы и хранения образов;

2) программные зависимости: установленная ОС Ubuntu Server 22.04 LTS с обновлёнными пакетами;

3) необходимое ПО: установка компонентов, которые являются зависимостями для Harbor:

— оболочка Docker Engine и Docker Compose;

— генератор OpenSSL.

Шаги установки следуют официальной документации проекта [16].

1) загрузка установщика: архив с установщиком Harbor загружается с официального GitHub-репозитория. Рекомендуется выбирать последнюю стабильную версию;

2) настройка конфигурации: основные параметры системы задаются в файле ‘harbor.yml‘. Минимальная конфигурация для тестового запуска включает:

— задание сетевого имени или IP-адреса хоста (‘hostname‘);

— настройка пароля для административного интерфейса (‘harbor_admin_password‘);

— отключение HTTPS для упрощения первой установки (комментарий для раздела ‘https:‘, активация порта 80 в разделе ‘http:‘).

3) запуск скрипта установки: выполнение подготовительного скрипта, который загружает необходимые образы контейнеров и генерирует конфигурации.

В процессе работы скрипт проверит наличие зависимостей, загрузит образы контейнеров Harbor и запустит все службы через Docker Compose.

После успешного выполнения скрипта система будет доступна:

1) через веб-интерфейс: по адресу `http://<IP_СЕРВЕРА>`. Логин: `admin`, пароль: заданный в ‘harbor.yml‘;

2) через Docker CLI: для работы с реестром необходимо добавить его адрес в конфигурацию Docker (для тестовой установки без HTTPS).

Данные шаги позволят полностью развернуть Harbor на операционной системе на основе UNIX.

В листингах 1.2–1.3 представлены основные команды для корректной установки и развёртывания Harbor.

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y docker.io docker-compose-v2 openssl
sudo systemctl enable docker && sudo systemctl start docker
```

Листинг 1.1 — Установка базовых зависимостей на Ubuntu Server.

```
wget https://github.com/goharbor/harbor/releases/download/v2.10.0/
   harbor-online-installer-v2.10.0.tgz
tar xzf harbor-online-installer-v2.10.0.tgz
cd harbor
```

Листинг 1.2 — Загрузка и распаковка установщика Harbor.

```
echo '{"insecure-registries": ["<IP>"]}' | sudo tee /etc/docker/
   daemon.json
sudo systemctl restart docker
```

Листинг 1.3 — Настройка Docker для работы с небезопасным реестром (для теста).

Дальнейшие шаги по настройке интегрированного сканирования уязвимостей, репликации, подключения внешних систем аутентификации (LDAP/OIDC) и настройки политик хранения выполняются через веб-интерфейс или путём редактирования конфигурационных файлов, что подтверждает гибкость и готовность Harbor к использованию в корпоративной среде [4].

2 Существующие решения

2.1 Критерии сравнения систем управления реестром

На основе анализа современных тенденций контейнеризации и изучения возможностей системы Harbor как эталонного решения можно сформулировать комплекс требований к корпоративной системе управления реестром Docker-образов. Эти требования охватывают различные аспекты функционирования системы и будут использованы в качестве основы для сравнительного анализа существующих решений.

В области безопасности система должна обеспечивать многоуровневую защиту артефактов и инфраструктуры. Обязательным является наличие интегрированных механизмов сканирования уязвимостей с регулярным обновлением баз данных CVE и возможностью интеграции с различными сканерами. Система должна поддерживать цифровую подпись образов для гарантии их целостности и аутентичности, предотвращая использование скомпрометированных артефактов в production-среде. Гибкая система управления доступом на основе ролей (RBAC) с поддержкой интеграции с корпоративными системами аутентификации (LDAP, Active Directory, OIDC) позволяет централизованно управлять правами доступа в соответствии с организационной структурой [16].

С точки зрения управления жизненным циклом артефактов, корпоративная система должна предоставлять инструменты для эффективного версионирования и тегирования образов, включая поддержку семантического версионирования и тегов [2]. Автоматическое управление политиками хранения с возможностью настройки правил удаления устаревших образов на основе различных критериев (время, количество версий, теги) необходимо для оптимизации использования дискового пространства. Поддержка квот и ограничений на уровне проектов и пользователей предотвращает неконтролируемый рост объема данных и обеспечивает равномерное распределение ресурсов.

Для обеспечения высокой доступности и отказоустойчивости система должна предоставлять механизмы репликации между географически распределенными инстансами, поддерживая как синхронные, так и асинхронные режимы синхронизации. Возможности горизонтального масштабирования и балансировки нагрузки позволяют системе адаптироваться к росту числа пользователей и объема данных без потери производительности [12]. Регулярное резервное копирование метаданных и образов с возможностью быстрого восстановления после сбоев является критически важным требованием для обеспечения непрерывности бизнес-процессов.

Интеграционные возможности системы должны обеспечивать взаимодействие с существующей инфраструктурой организации. Поддержка REST API для автоматизации процессов CI/CD позволяет интегрировать реестр в процессы непрерывной интеграции и доставки, ускоряя разработку и повышая ее надежность. Современный веб-интерфейс с интуитивной навигацией и комплексными возможностями управления упрощает ежедневную работу администраторов и разработчиков, снижая порог вхождения и уменьшая вероятность ошибок. Наличие инструмен-

тов командной строки и программных интерфейсов обеспечивает возможности для автоматизации задач управления и интеграции со скриптами и системами оркестрации.

Сформулированные требования представляют собой комплексный набор критериев, который будет использован для проведения сравнительного анализа систем управления реестрами контейнерных образов. Соответствие этим требованиям позволит объективно оценить пригодность различных решений для использования в корпоративной среде:

1) сканирование уязвимостей - возможность автоматического обнаружения уязвимостей в контейнерных образах;

2) цифровая подпись образов - поддержка механизмов гарантии целостности и аутентичности артефактов;

3) RBAC - управление доступом на основе ролей с интеграцией корпоративных систем аутентификации;

4) репликация - возможности синхронизации данных между георасределенными инстансами;

5) политики хранения - автоматическое управление жизненным циклом артефактов;

6) квоты - установка ограничений на использование ресурсов;

7) поддержка различных артефактов - работа не только с Docker-образами, но и с Helm-чартами, OCI-артефактами;

8) интеграция с CI/CD - возможности взаимодействия с системами непрерывной интеграции и доставки;

9) веб-интерфейс (UI) - наличие пользовательского интерфейса для управления реестром;

10) API и CLI - программные интерфейсы для автоматизации задач управления.

2.2 Результаты сравнения

Результаты комплексного сравнения 30 систем управления реестрами Docker-образов представлены в таблицах приложения А. Таблица демонстрирует уровень поддержки ключевых функций в различных категориях решений.

Для каждого параметра использовалась трехбалльная система оценки:

— 2 - функция полностью поддерживается;

— 1 - функция поддерживается частично или требует дополнительной настройки;

— 0 - функция отсутствует.

2.3 Анализ полученных результатов

Для наглядного представления уровня поддержки ключевых функций среди рассмотренных систем были построена аналитическая диаграмма.

На рисунке 2.1 представлена диаграмма сравнения поддержки ключевых функций.

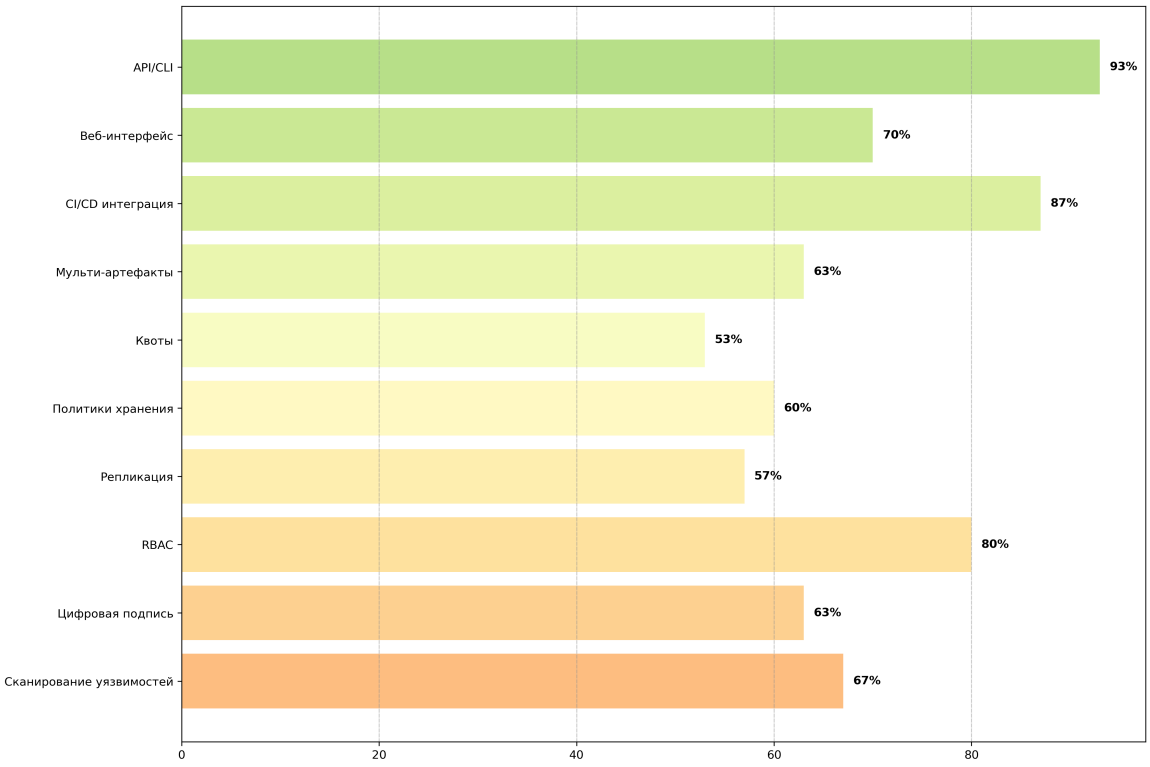


Рисунок 2.1 — Процент поддержки ключевых функций среди 30 систем управления реестрами

На основе проведенного анализа можно сформулировать следующие рекомендации:

1) для организаций, ищущих открытое решение с полным функционалом, оптимальным выбором является Harbor;

2) при использовании конкретного облачного провайдера целесообразно рассматривать его нативные сервисы (ECR, ACR, GCR);

3) для крупных предприятий с комплексными требованиями подходят коммерческие решения JFrog Artifactory, Red Hat Quay Enterprise;

4) Базовые решения Docker Registry пригодны только для тестовых сред и разработки

Проведенное исследование подтвердило, что система Harbor является эталонным решением в категории open-source систем управления реестрами и может служить ориентиром при оценке других решений. Ее функциональность полностью покрывает требования корпоративной среды, что делает ее оптимальным выбором для организаций, seeking баланс между функциональностью, гибкостью и стоимостью владения.

2.4 Интеграция Harbor в учебный процесс кафедры

Система Harbor может быть интегрирована в учебный процесс для автоматизации проверки лабораторных работ и студенческих проектов. Практика создания тестовых пайплайнов уже давно применяется, однако интеграция системы Harbor позволит оптимизировать данный процесс.

Архитектура интеграции Harbor в учебный процесс включает несколько компонентов. Git-репозиторий содержит задания лабораторных работ, Dockerfile для сборки и тесты. Система

Harbor выполняет роль приватного реестра для хранения базовых образов, образов с решениями студентов и тестовых образов. CI/CD система автоматизирует процессы сборки, тестирования и развертывания. Система мониторинга собирает результаты тестирования и метрики выполнения.

2.4.1 Сценарий использования для лабораторных работ

Процесс выполнения лабораторной работы начинается с того, что студент получает доступ к Git-репозиторию с заданием. Репозиторий содержит описание лабораторной работы, Dockerfile для сборки образа приложения, тесты для проверки корректности решения и конфигурационные файлы для CI/CD.

Студент разрабатывает решение и размещает его в личной ветке Git-репозитория. При отправке изменений запускается CI/CD пайплайн. Пайплайн собирает Docker-образ из решения студента, загружает образ Harbor в личный репозиторий студента, после чего запускает тестовый контейнер. Тестовый контейнер скачивает образ решения из Harbor, выполняет тесты в изолированном окружении и генерирует отчет о выполнении. Результаты тестирования публикуются в системе мониторинга.

Преподаватель получает доступ к результатам через веб-интерфейс Harbor.

2.4.2 Пример реализации CI/CD пайплайна

Для интеграции Harbor с GitLab CI может использоваться конфигурационный файл .gitlab-ci.yml. В листинге 2.1 представлен пример конфигурации.

```
stages:
- build
- test
- deploy

variables:
HARBOR_URL: "harbor.bmstu.cloud"
PROJECT: "iu7-labs"
IMAGE_NAME: "lab1-app"

build:
stage: build
image: docker:latest
services:
- docker:dind
script:
- docker build -t ${HARBOR_URL}/${PROJECT}/${IMAGE_NAME}:${
   CI_COMMIT_SHA} .
- docker login -u ${HARBOR_USER} -p ${HARBOR_PASSWORD} ${HARBOR_URL}
- docker push ${HARBOR_URL}/${PROJECT}/${IMAGE_NAME}:${CI_COMMIT_SHA}
only:
- branches

test:
stage: test
image: docker:latest
services:
- docker:dind
script:
- docker login -u ${HARBOR_USER} -p ${HARBOR_PASSWORD} ${HARBOR_URL}
- docker pull ${HARBOR_URL}/${PROJECT}/${IMAGE_NAME}:${CI_COMMIT_SHA}
- docker run --rm ${HARBOR_URL}/${PROJECT}/${IMAGE_NAME}:${
   CI_COMMIT_SHA} python -m pytest tests/
artifacts:
reports:
junit: test-results.xml
```

Листинг 2.1 — Пример .gitlab-ci.yml для лабораторной работы

В этом примере определены три стадии: сборка, тестирование и развертывание. На стадии сборки создается Docker-образ из исходного кода студента, который затем загружается в Harbor. На стадии тестирования образ скачивается из Harbor и запускаются тесты с использованием pytest. Результаты тестирования сохраняются в формате JUnit для последующего анализа.

ЗАКЛЮЧЕНИЕ

Не существует универсальной системы управления реестром Docker-образов, которая подходила бы для всех сценариев использования. Разные решения имеют свои сильные и слабые стороны, поэтому выбор конкретной системы определяется поставленными задачами и требованиями организации.

Как показало исследование, для корпоративных сред важны такие функции как безопасность, отказоустойчивость и интеграция с существующей инфраструктурой. В таких случаях можно выбирать из решений уровня Harbor, Quay или облачных сервисов крупных провайдеров. Однако для тестовых сред или небольших проектов могут подойти и более простые решения.

В данной научно-исследовательской работе был:

1) проведен анализ предметной области управления реестрами Docker-образов;
2) выполнена классификация задач и требований к системам управления реестрами;
3) проведен сравнительный анализ 30 систем по сформулированным критериям;
4) определены преимущества и ограничения рассмотренных решений.

В рамках поставленной работы были выполнены все задачи. Цель работы — проведение сравнительного анализа систем управления реестрами на основе платформы Harbor — достигнута.

На основе проведенного исследования можно сделать вывод, что система Harbor является одним из наиболее сбалансированных решений среди открытых систем и может быть рекомендована для организаций, ищущих полнофункциональную платформу без привязки к конкретному облачному провайдеру. Однако окончательный выбор системы должен учитывать конкретные требования и контекст использования.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Harbor: The open source trusted cloud native registry [Электронный ресурс]. Режим доступа: https://goharbor.io/docs/ (Дата обращения: 10.11.2025)
2. Docker Registry HTTP API V2 [Электронный ресурс]. Режим доступа: https://docs.docker.com/registry/spec/api/ (Дата обращения: 10.11.2025)
3. OCI Distribution Specification [Электронный ресурс]. Режим доступа: https://github.com/opencontainers/distribution-spec (Дата обращения: 13.11.2025)
4. CNCF Cloud Native Landscape [Электронный ресурс]. Режим доступа: https://landscape.cncf.io/card-mode?category=container-registry (Дата обращения: 13.11.2025)
5. Banyan, M. Comparative Analysis of Private Docker Registries: Harbor vs. Quay // Journal of Cloud Computing. 2022. №4. С. 45-62.
6. Zhang, L. Security Analysis of Container Image Registries in CI/CD Pipelines // IEEE Transactions on Dependable and Secure Computing. 2021. №3. С. 1125-1138.
7. Cilic, I. Krivic, P. Podnar Zarko, I. Kusek, M. Performance Evaluation of Container Orchestration Tools inEdge Computing Environments // Sensors. 2023. 23 с.
8. Container Registry Security Best Practices [Электронный ресурс]. Режим доступа: https://www.aquasec.com/cloud-native-academy/container-security/container-security-best-practices/ (Дата обращения: 13.11.2025)
9. Software Supply Chain Security // Cloud Native Security Paper. 2022. 42 с.
10. Иан Милл, Эйдан Хобсон Сейерс Docker на практике / пер. с англ. Д.А. Беликов. – М.: ДМК Пресс, 2020. – 516 с.
11. Марко Лукша Kubernetes в действии / пер. с англ. А. В. Логунов. – М.: ДМК Пресс, 2019. – 672 с
12. Market Guide for Container Management // Gartner Technical Report. 2023. 54 с.
13. The Forrester Wave™: Enterprise Container Platform // Forrester Research Report. 2022. 48 с.
14. Container Registry Usage and Trends // Developer Economics Survey. 2023. 67 с.
15. Cloud Native Security and Usage Report // Annual Container Security Report. 2024. 21 с.
16. Harbor Documentation [Электронный ресурс]. Режим доступа: https://goharbor.io/docs/ (Дата обращения: 20.11.2025)
17. Docker Registry Documentation [Электронный ресурс]. Режим доступа: https://docs.docker.com/registry/ (Дата обращения: 20.11.2025)
18. Red Hat Quay Documentation [Электронный ресурс]. Режим доступа: https://access.redhat.com/documentation/red-hat-quay (Дата обращения: 14.11.2025)

19. Sonatype Nexus Documentation [Электронный ресурс]. Режим доступа: https://help.sonatype.com/repomanager3 (Дата обращения: 14.11.2025)

20. JFrog Artifactory Documentation [Электронный ресурс]. Режим доступа: https://www.jfrog.com/confluence/display/JFROG/JFrog+Artifactory (Дата обращения: 14.11.2025)

21. GitLab Container Registry Documentation [Электронный ресурс]. Режим доступа: https://docs.gitlab.com/ee/user/packages/container_registry/ (Дата обращения: 14.11.2025)

22. Amazon ECR Documentation [Электронный ресурс]. Режим доступа: https://docs.aws.amazon.com/ecr/ (Дата обращения: 20.11.2025)

23. Azure Container Registry Documentation [Электронный ресурс]. Режим доступа: https://docs.microsoft.com/azure/container-registry/ (Дата обращения: 15.11.2025)

24. Google Container Registry Documentation [Электронный ресурс]. Режим доступа: https://cloud.google.com/container-registry/docs (Дата обращения: 15.11.2025)

25. GitHub Container Registry Documentation [Электронный ресурс]. Режим доступа: https://docs.github.com/packages/working-with-a-github-packages-registry/working-with-the-container-registry (Дата обращения: 15.11.2025)

26. IBM Cloud Container Registry Documentation [Электронный ресурс]. Режим доступа: https://cloud.ibm.com/docs/Registry (Дата обращения: 18.11.2025)

27. DigitalOcean Container Registry Documentation [Электронный ресурс]. Режим доступа: https://docs.digitalocean.com/products/container-registry/ (Дата обращения: 18.11.2025)

28. Oracle Container Registry Documentation [Электронный ресурс]. Режим доступа: https://docs.oracle.com/iaas/Content/Registry/Concepts/registryoverview.htm (Дата обращения: 18.11.2025)

29. Portus Documentation [Электронный ресурс]. Режим доступа: https://github.com/SUSE/portus (Дата обращения: 18.11.2025)

30. Rescoyl Documentation [Электронный ресурс]. Режим доступа: https://github.com/reshufflehq/rescoyl (Дата обращения: 19.11.2025)

31. Kraken Documentation [Электронный ресурс]. Режим доступа: https://github.com/uber/kraken (Дата обращения: 20.11.2025)

32. OCI Distribution Specification [Электронный ресурс]. Режим доступа: https://github.com/opencontainers/distribution-spec (Дата обращения: 20.11.2025)

33. Docker Hub Documentation [Электронный ресурс]. Режим доступа: https://docs.docker.com/docker-hub/ (Дата обращения: 20.11.2025)

34. Clear Linux Registry [Электронный ресурс]. Режим доступа: https://docs.01.org/clearlinux/latest/guides/container/registry.html (Дата обращения: 20.11.2025)

35. K3s Integrated Registry [Электронный ресурс]. Режим доступа: https://docs.k3s.io/installation/private-registry (Дата обращения: 20.11.2025)

36. Mirror Registry for OpenShift [Электронный ресурс]. Режим доступа: https://docs.openshift.com/container-platform/4.12/registry/registry-mirror.html (Дата обращения: 20.11.2025)

37. Rancher Registry [Электронный ресурс]. Режим доступа: https://rancher.com/docs/rancher/v2.6/en/helm-charts/registries/ (Дата обращения: 20.11.2025)

38. Portainer Business Registry [Электронный ресурс]. Режим доступа: https://docs.portainer.io/user/registries (Дата обращения: 21.11.2025)

39. AQUA Security Registry [Электронный ресурс]. Режим доступа: https://docs.aquasec.com/docs/container-registries (Дата обращения: 21.11.2025)

40. Sysdig Secure Registry [Электронный ресурс]. Режим доступа: https://docs.sysdig.com/en/docs/installation/sysdig-secure/scanning/registries/ (Дата обращения: 23.11.2025)

41. Podman Local Registry [Электронный ресурс]. Режим доступа: https://docs.podman.io/en/latest/markdown/podman-run.1.html (Дата обращения: 23.11.2025)

42. MicroK8s Registry [Электронный ресурс]. Режим доступа: https://microk8s.io/docs/registry-built-in (Дата обращения: 23.11.2025)

43. KIND Registry [Электронный ресурс]. Режим доступа: https://kind.sigs.k8s.io/docs/user/local-registry/ (Дата обращения: 23.11.2025)

44. Carina Registry from Rancher [Электронный ресурс]. Режим доступа: https://github.com/rancher/carina (Дата обращения: 23.11.2025)

45. Docker Registry UI [Электронный ресурс]. Режим доступа: https://github.com/Joxit/docker-registry-ui (Дата обращения: 23.11.2025)

ПРИЛОЖЕНИЕ А

Результаты комплексного сравнения 30 систем управления реестрами Docker-образов.

Таблица А.1 — Сравнительный анализ систем управления реестрами Docker-образов (системы 1-10)

№	Система	Критерии оценки (0-2)									
		1	2	3	4	5	6	7	8	9	10
1	Harbor	2	2	2	2	2	2	2	2	2	2
2	Docker Registry	0	0	0	0	0	0	1	0	1	2
3	Red Hat Quay	2	2	2	1	1	1	2	2	2	2
4	JFrog Artifactory	2	2	2	1	2	2	2	2	2	2
5	GitLab Container Registry	1	0	2	0	1	0	1	2	2	2
6	Amazon ECR	2	1	2	2	1	2	2	2	2	2
7	Azure Container Registry	2	1	2	2	1	2	2	2	2	2
8	Google Container Registry	1	0	2	0	1	1	2	2	2	2
9	Nexus Repository Manager	1	0	2	1	2	2	2	2	2	2
10	Portus	1	1	2	0	1	0	1	1	2	1

Таблица А.2 — Сравнительный анализ систем управления реестрами Docker-образов (системы 11-20)

№	Система	Критерии оценки (0-2)									
		1	2	3	4	5	6	7	8	9	10
11	IBM Cloud Container Registry	1	0	2	2	1	2	2	2	2	2
12	DigitalOcean Container Registry	1	0	2	0	1	1	2	2	2	2
13	Oracle Container Registry	1	0	2	2	1	2	2	2	2	2
14	Docker Hub	1	0	2	0	1	0	2	2	2	2
15	Clear Linux Registry	0	0	0	0	0	0	1	0	1	1
16	K3s Integrated Registry	0	0	0	0	0	0	1	0	1	1
17	Mirror Registry for OpenShift	2	1	2	1	1	1	2	2	2	2
18	Rancher Registry	1	0	2	0	1	0	2	2	2	2
19	Portainer Business Registry	1	0	2	0	1	0	2	2	2	2
20	AQUA Security Registry	2	2	2	2	2	2	2	2	2	2

Таблица А.3 — Сравнительный анализ систем управления реестрами Docker-образов (системы 21-30)

№	Система	Критерии оценки (0-2)									
		1	2	3	4	5	6	7	8	9	10
21	Sysdig Secure Registry	2	2	2	2	2	2	2	2	2	2
22	Podman Local Registry	0	0	0	0	0	0	1	0	0	2
23	MicroK8s Registry	0	0	0	0	0	0	1	0	0	1
24	KIND Registry	0	0	0	0	0	0	1	0	0	1
25	Carina Registry from Rancher	1	0	2	0	1	0	2	2	2	2
26	Docker Registry UI	0	0	0	0	0	0	1	0	2	0
27	Rescoyl	1	1	2	0	1	0	1	1	2	1
28	Kraken	1	0	2	2	1	1	2	2	1	2
29	GitHub Container Registry	1	0	2	0	1	0	2	2	2	2
30	Sonatype Nexus Repository	2	1	2	1	2	2	2	2	2	2

ПРИЛОЖЕНИЕ Б

Презентация к научно-исследовательской работе.